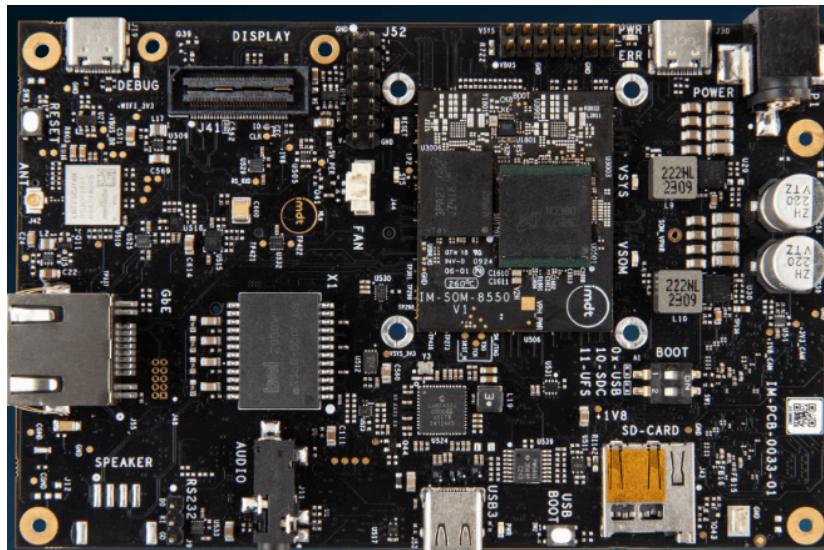


IMDT SBC8550 BSP

Getting Started Guide



Revision History	4
References	4
Introduction	5
Prerequisites	5
Common Environment Variables	5
Using the prebuilt release	6
Building release images	7
Host Setup	7
Enable CodeLinaro access and Download Qualcomm Sources	7
Load docker image	8
Setting up a container based on a docker image	8
Container Environment Setup	9
Setup environment environment variables	9
Create .netrc file	9
Synchronise Repos	10
Configure Kernel Directories	11
Apply IMDT patches to Qualcomm source	11
Building Images	12
Build Linux Embedded (LE) Kernel	12
Build HLOS (High Level Operating System) components	13
Generate SWUpdate (SWU) Image	14
Generate Yocto SDK	14
Build non-HLOS components	15
Generate Configuration Data Table (CDT) Image	16
Deploying images	17
Install Qualcomm Package Manager 3 (QPM3)	17
QPM CLI Login	17
Activate product licences	18
Install QUTS, QUD and PCAT	18
Disabling Modem Manager	18
Boot SBC into Emergency Download (EDL) mode	19
Read Device Serial Number	19
Flashing Configuration Data Table Image	20
Flashing Software Images	20
Working with the board	21
Powering the Board	21
Connecting to the Debug Serial Consoles	21
Android Debug Bridge (ADB) and Fastboot	22
Setting up ADB and Fastboot	22
Install ADB and Fastboot on host machine	22

Check connected devices with ADB	22
Development Workflow using ADB	22
Remount the root directory for RW access	22
Copying files onto device	22
Launch ADB Shell	22
Flashing using Fastboot	23
Reboot device into Fastboot Mode	23
Flash whole image onto device using Fastboot Protocol	23
Using the Display	24
Streaming from MIPI Camera's	25
Streaming from AR1335	25
Audio	26
SD-Card	26
Using the IMU (Accelerometer and Gyroscope)	26
Using the Temperature and Humidity Sensor	27
Qualcomm's Intelligent Multimedia SDK	28
Downloading Qualcomm's GStreamer AI Models	28
Downloading Base Models and Labels	28
Downloading YOLOv8 model	30
Running Qualcomm's GStreamer AI Sample Applications	31
Initialisation	31
Classification	31
Daisychain Detection Pose	31
Multistream Inference	31
Object Detection	31
Parallel Inference	31
Pose Detection	31
Networking	32
Gigabit Ethernet over RJ45	32
DHCP Client	32
DHCP Server	32
Onboard WiFi	33
WiFi via M2.E WiFi card	35
BSP OTA Software Update	37
Using the SDK	40
Installing the SDK	40
Setting up a Development Environment	41
Building a "Hello World" Application Using the SDK	41
Running the Example Application on the board	42
Appendix	43

1.1. Setting up a new user within docker	43
1.2. Connectors - Top Side	44
1.3. Connectors - Bottom Side	46

Revision History

Revision	Date	Author	Description
1	April 18th, 2025	Dimitrios Kapsoulis	Initial revision (V1.0.0 release)
2	April 30th, 2025	Ram B.	Updated images from HW
3	May 19th, 2025	Tendai Makumire	Added build process
4	June 2nd, 2025	Tendai Makumire	Updated the docker volume mount paths and added a section for creating a new user within the docker when there is a mismatch between docker and host UID:GID.
5	August 28th, 2025	Joseph Connan	Added commands for running the GStreamer AI sample applications
6	September 9th, 2025	Thomas Bourlard	Added section on IMU, Ethernet & WiFi.
7	January 2nd, 2026	Dimitrios Kapsoulis	Added section on Temperature and Humidity Sensor, Onboard Wifi and SWUpdate.
8	February 2nd, 2026	Dimitrios Kapsoulis	Added section on Audio and SD-Card

References

- [1] [IMDT - QCS8550 SBC Datasheet rev 1.2.pdf](#)
- [2] [IMDT SBC8550 - SWUpdate.pdf](#)

Introduction

This guide describes how to build, deploy and run applications on the IMDT QCS8550 SBC.

Prerequisites

This process has been tested on machines with the following parameters.

- Ubuntu 18.04, 20.04 or 22.04
- Docker Engine v24.02
- 8 Core / 16 Threads Processor
- 64GiB RAM
- At least 500GB of disk space on an `ext4` formatted drive

Common Environment Variables

In order to make code snippets more generic a number of environment variables are throughout this document. These must be set before carrying out any instructions that use them. They are:

```
# BSP_VERSION should be set to the BSP version. So, for example, for QCS8550-SBC-v2.0.0 the version would be set to 2.0.0
```

```
$ BSP_VERSION=version
```

```
# BUILD_VERSION should be set to the build version specified in the release notes
```

```
# This should match the version number of the SBC prebuilt release image
```

```
$ BUILD_VERSION=version
```

```
# DOCKER_VERSION should be set to the correct docker version for the BSP
```

```
$ DOCKER_VERSION=version
```

```
# BSP_DIR should be set to the path to the IMDT QCS8550 SBC BSP directory
```

```
$ BSP_DIR=/path/to/QCS8550-SBC-BSP-v${BSP_VERSION}
```

Note that the version numbers are defined in the Release Notes associated with each BSP release.

Using the prebuilt release

The prebuilt release can be directly flashed onto the board without following the build process. This can be used along with the SDK for jumping straight into development.

To extract the prebuilt release, do the following:

```
$ cd ${BSP_DIR}
$ tar -xf ./imdt_qcs8550_sbc_prebuilt_release_v${BUILD_VERSION}.tar.gz
```

To flash the board, with the extracted prebuilt release, follow the steps outlined in [Flashing Software Images](#) or [Flashing using Fastboot](#). When following the instructions outlined in those sections, make sure that the prebuilt release folder is used instead of the source release folder.

Building release images

This section describes the process for building the release images from source.

Host Setup

Enable CodeLinaro access and Download Qualcomm Sources

Fetching sources from CodeLinaro needs to be enabled through Qualcomm ChipCode for the specific release that is being built. For **QCS8550-SBC-v\${BSP_VERSION}**, CodeLinaro access for **release r00087.1** needs to be enabled. To accomplish this, go to [Qualcomm ChipCode](#), search QCS8550.LE.1.0 in the Qualcomm ChipCode search bar and then click on the repo named **qcs8550-le-1-0_amss_standard_oem_apqgps**. Select **release r00087.1** as the release tag, then click *Enable CodeLinaro* for each CodeLinaro repo listed in the release.

QCS8550.LE.1.0 / qcs8550-le-1-0_amss_standard_oem_apqgps / All customers

Size 12404.45 MB

[qcs8550-le-1-0_amss_standard_oem_apqgps](#) ☆

Choose a specific Release Tag, or select a Branch to view the latest Release on that branch.

☐ Hide Service Releases

r00011.1

Released at Thu May 04 2023 18:58:39 GMT-0700 (PDT)

[View Release History](#)

You are about to download software that may not contain Qualcomm Technologies' (QTI) most up to date software patches. We strongly recommend that you review QTI's most current patch releases by using the procedure specified [80-P2484-38 Android Security Bulletin Support](#). Electing to not use QTI's most current patches is at your own risk. QTI is not liable for any issues which may arise due to your failure to implement the most current patch releases. If you need additional support which is currently outside of your current support agreement with QTI, please contact your QTI sales representative.

Clone the repository

git clone https://qpm-git.qualcomm.com/home2/git/qualcomm/qcs8550

GZip ☒ Zip

[Download Release r00011.1](#)

Clone this release

git clone -b r00011.1 --depth 1 https://qpm-git.qualcomm.com/home2/git/

CodeLinaro: KERNEL PLATFORM 2.0.R11	repo init --depth=1 --current-branch -u https://git.codelinaro.org/	🔗	May 12, 2023 02:56 AM	r00011.1	FC	Enabled CodeLinaro
CodeLinaro: LE.FRAMEWORK 2.0.R1	repo init --depth=1 --current-branch -u https://git.codelinaro.org/clo/le/	🔗	May 12, 2023 02:56 AM	r00011.1	FC	Enabled CodeLinaro
CodeLinaro: LE.PRODUCT 2.1.R1	repo init --depth=1 --current-branch -u https://git.codelinaro.org/clo/le/le/	🔗	May 12, 2023 02:56 AM	r00011.1	FC	Enabled CodeLinaro

[Click Here](#)

After enabling CodeLinaro access, click *Download Release* (Make sure to download it as a GZip and not a Zip). The release is quite big so it takes a while for Qualcomm's servers to compress the release. The release can also be cloned via Git but this doesn't always work. Success of the Git clone is highly dependent on the internet connection and the Git version being used.

Also make sure to clone/download the Qualcomm release into \$BSP_DIR. It is referred to as \$QCS_SOURCES in the rest of the document.

Load docker image

```
$ cd $BSP_DIR
$ docker load < imdt-qualcomm-build-setup_${DOCKER_VERSION}.tar.xz
```

Setting up a container based on a docker image

Once the docker image has been downloaded onto the host machine, a container will need to be created for building and development.

The *docker run* command is used to start a container based on an image. And *docker attach* is used to enter a running container. The table below shows the different options that are used with *docker run*.

Option	Description	Arguments
-it	Launches interactive shell in container	N/A
-d	Launches container as background process	N/A
--rm	Deletes container on exit	N/A
-v	Mount a volume on host machine into container	-v /path/on/host/./path/in/container
--name	Tags the container with a name	--name my_container

To run the docker container, use the following:

```
$ cd $BSP_DIR
$ tar -xf ${QCS_SOURCES}.tar.gz -C ${BSP_DIR}

$ docker run -it -d --rm -v ${BSP_DIR}:/Qualcomm --name qualcomm-build-env
imdttec/imdt-qualcomm-build-setup:${DOCKER_VERSION}

$ docker attach qualcomm-build-env
```


Container Environment Setup

This section describes how to set up the environment *within* the container after attaching to it. **Note that if the host UID and GID does not match 1000:1000, then a new user will need to be created within the docker that matches the host UID and GID.** See [Setting up a new user within docker](#) in the [Appendix](#).

Setup environment environment variables

```
$ QCOM_ROOT_DIR=/path/to/qcs/sources/folder
```

For example:

```
$ export
QCOM_ROOT_DIR=/Qualcomm/qcs8550-1e-1-0_amss_standard_oem_apqgps.git
```

Create .netrc file

The .netrc file must be filled out with the login credentials for Qualcomm Chipcode, CodeLinaro and GitHub. The login details for Qualcomm chipcode are just the login details used for logging into Qualcomm.com.

For CodeLinaro access, a Personal Access Token must be generated through CodeLinaro's GitLab interface. For details on how to set up accounts for Qualcomm ChipCode and CodeLinaro see document 80-35346-1 (Open Source Release Process Change).

```
$ echo "machine git.codelinaro.org" >> ~/.netrc
$ echo "login CODE_LINARO_LOGIN" >> ~/.netrc
$ echo "password CODE_LINARO_PERSONAL_ACCESS_TOKEN" >> ~/.netrc
$ echo "machine chipcode.qti.qualcomm.com" >> ~/.netrc
$ echo "login QCOM_CHIPCODE_LOGIN" >> ~/.netrc
$ echo "password QCOM_CHIPCODE_PASSWORD" >> ~/.netrc
$ echo "machine chipmaster2.qti.qualcomm.com" >> ~/.netrc
$ echo "login QCOM_CHIPCODE_LOGIN" >> ~/.netrc
$ echo "password QCOM_CHIPCODE_PASSWORD" >> ~/.netrc
$ echo "machine github.com" >> ~/.netrc
$ echo "login GITHUB_LOGIN" >> ~/.netrc
$ echo "password GITHUB_PERSONAL_ACCESS_TOKEN" >> ~/.netrc
$ chmod og-rw ~/.netrc
```

Synchronise Repos

This step involves fetching the Linux kernel and the relevant meta-layers from the upstream sources using a repo manifest. IMDT provides its own repo manifest alongside Qualcomm's which will fetch the IMDT specific meta-layers.

To synchronise the repos as described above use the following:

```
$ tar -xf /Qualcomm/patches.tar.gz -C /Qualcomm/

$ patch ${QCOM_ROOT_DIR}/LE.PRODUCT.2.1.r1/apps_proc/sync_snap_v2.sh
~/Qualcomm/patches/sync_snap_v2_remove_chipcode_copy.patch

$ ~/build_scripts/sync_repos.sh -u
https://github.com/imd-tec/imdt-qcom-manifest.git -b kirkstone -m
imdt-qcom-bsp-v${BUILD_VERSION}.xml
```

To get more information about the options provided by the *sync_repos* shell script use the following:

```
$ ~/build_scripts/sync_repos.sh --help
```

Configure Kernel Directories

After all the sources have been fetched, the kernel directories need to be set up. There is a simple helper script that can be used to do this automatically.

```
$ ~/build_scripts/setup_kernel.sh
```

Apply IMDT patches to Qualcomm source

Once the kernel directories have been set up, the source now needs to be patched with the changes required to run on the IMDT QCS8550 SBC.

To patch the Qualcomm source, do the following:

```
$ cd ${QCOM_ROOT_DIR}/LE.PRODUCT.2.1.r1/apps_proc  
$ ./imdt-patch-qcs8550-build.sh
```

Building Images

The following section describes how to build an image for the SBC within the docker container.

Build Linux Embedded (LE) Kernel

After successfully syncing the repos, the Linux Kernel and accompanying kernel modules may be built. Although Qualcomm uses Yocto in their build system, the Linux kernel is built outside of Yocto. The build outputs of the kernel are used as the build inputs to the Yocto build (i.e. building the HLOS components).

To build the kernel, call the *build_kernel* shell script like this:

```
$ ~/build_scripts/build_kernel.sh --lto "thin" --jobs 8
```

To help with kernel development, this script can also be used to do incremental builds with different build configurations. To get more information about how to use this script, use the help menu with the following:

```
$ ~/build_scripts/build_kernel.sh --help
```

Build HLOS (High Level Operating System) components

Building the HLOS components involves using bitbake to build the packages that go into the root filesystem (rootfs). Before these packages can be built, some Qualcomm files in the *LE.FRAMEWORK* folder must be copied over to *LE.PRODUCT* using the following:

```
$ cp -rn ${QCOM_ROOT_DIR}/LE.FRAMEWORK.2.0.r1/apps_proc/poky/*  
${QCOM_ROOT_DIR}/LE.PRODUCT.2.1.r1/apps_proc/poky/  
  
$ cp -rn ${QCOM_ROOT_DIR}/LE.FRAMEWORK.2.0.r1/apps_proc/src/*  
${QCOM_ROOT_DIR}/LE.PRODUCT.2.1.r1/apps_proc/src/
```

Then to build the HLOS components including the rootfs, setup the Yocto environment and build the image:

```
$ cd ${QCOM_ROOT_DIR}/LE.PRODUCT.2.1.r1/apps_proc  
$ pyenv global 3.8.16  
  
# Setup Yocto environment  
$ export MACHINE=imdt-qcs8550-sbc  
$ export DISTRO=imdt-qcom-distro-debug  
$ source poky/qti-conf/set_bb_env.sh
```

Build the image:

```
# Fetch all sources related to the image  
$ bitbake -k imdt-image-weston --runall=fetch  
$ bitbake -k imdt-image-weston
```

Generate SWUpdate (SWU) Image

The SWU image can be used to update the operating system.

To generate the image, execute the following:

```
$ bitbake imdt-image-weston-swu
```

The image (*imdt-image-weston-swu-imdt-qcs8550-sbc.swu*) will be located inside the build folder

LE.PRODUCT.2.1.r1/apps_proc/build-imdt-qcom-distro-debug/tmp-glibc/deploy/images/imdt-qcs8550-sbc. More information regarding the SWUpdate application and how it works can be found in the [guide](#).

Generate Yocto SDK

The Yocto SDK can be used to cross compile and debug applications that run on the target device. To learn how to use the SDK, see [Using the SDK](#).

To generate a Linux Yocto SDK, execute the following:

```
$ bitbake imdt-image-weston -c populate_sdk
```

Note that this section assumes the Yocto build environment has already been set up in section [Build HLOS \(High Level Operating System\) components](#).

Build non-HLOS components

After building all the HLOS components, the non-HLOS components can be built. The list of components to be built includes:

eXtensible Bootloader (XBL) image, Application DSP (ADSP) image, Compute DSP (CDSP) image, TrustZone (TZ) Image, Secure Application (SA) Image, Always-on Processor (AOP) image and the META image.

All the images can be built using the *build_non_hlos* shell script with the following:

```
$ ~/build_scripts/build_non_hlos.sh --build all
```

If more control is needed or a specific image fails to build, this script can also build each individual component. To get more information about how to use this script, use the help menu with the following:

```
$ ~/build_scripts/build_non_hlos.sh --help
```

Generate Configuration Data Table (CDT) Image

```
$ tar -xf /Qualcomm/CDT.tar.gz -C /Qualcomm/  
$ ~/build_scripts/build_cdt.sh
```

The *build_cdt* shell script also provides some configuration options to generate other CDT images. These can be viewed through the options menu with the following command:

```
$ ~/build_scripts/build_cdt.sh --help
```


Deploying images

At the moment, the device cannot be flashed from within the docker container, so it has to be done from the host machine running **Ubuntu 18.04, Ubuntu 20.04 or Ubuntu 22.04**. To flash the device, Qualcomm provides the Product Configuration Assistant tool (PCAT), Qualcomm Unified Tool Service (QUTS) and the Qualcomm USB Driver (QUD). The instructions below will show how images are deployed to the SBC using the tools provided by Qualcomm.

Install Qualcomm Package Manager 3 (QPM3)

Log into [Qualcomm® Software Center](#) using the login credentials for Qualcomm.com. Then search for “qualcomm package manager 3” and select the target OS as Linux. Download the installer from there and use the following commands to install it.

```
$ sudo dpkg -i /path/to/QualcommPackageManager<VERSION>.Linux-x86.deb
```

QPM CLI Login

To download PCAT, QUTS and QUD, the user must be logged into the QPM with the account details for Qualcomm.com.

```
$ qpm-cli --login QCOM_USER_NAME QCOM_PASSWORD
```

It may take a few minutes for it to login and fetch the product catalogue. If it's taking too long, the QPM3 GUI Tool can also be used to install the relevant tools as well. *The QPM3 GUI can be launched through the standard Ubuntu application browser menu.*

Activate product licences

```
$ qpm-cli --license-activate quts  
$ qpm-cli --license-activate qud  
$ qpm-cli --license-activate pcat
```

Install QUTS, QUD and PCAT

```
$ qpm-cli --install quts  
$ qpm-cli --install qud  
$ qpm-cli --install pcat
```

Disabling Modem Manager

This may not be necessary if the host machine does not have the ModemManager service running. This is to stop the ModemManager configuring the Qualcomm device as a modem, if it sees the Qualcomm device as a modem.

```
$ sudo systemctl stop ModemManager.service
```

Boot SBC into Emergency Download (EDL) mode

To boot the SBC into EDL mode, do the following:

1. Power on the board via the DC Jack or the USB PD port
2. Press and *hold* the USB BOOT button
3. Press the Reset button for 1-2 seconds whilst the USB BOOT button is held
4. Release the USB BOOT button

Refer to [Connectors - Top Side](#) to see location of USB BOOT and Reset buttons

Read Device Serial Number

```
$ pcat -DEVICES
```

If the SBC is found, the output should look similar to below. The serial number will need to be logged so that it can be used for flashing.

```
build@bstl-buildsrv-1:~$ pcat -DEVICES
2024-09-26 09:24:18.249 INFO    ] Is Qualcomm USB host drivers running, TRUE
2024-09-26 09:24:18.250 INFO    ] Creating QUTS client for PCAT operation...
2024-09-26 09:24:18.377 INFO    ] QUTS client creation completed, Status = TRUE
2024-09-26 09:24:18.417 INFO    ] Searching devices in device manager, please wait for a moment...

+-----+-----+-----+-----+-----+-----+
| ID | DEVICE TYPE | DEVICE STATE | SERIAL NUMBER | ADB SERIAL NUMBER | DESCRIPTION |
+-----+-----+-----+-----+-----+-----+
| NA | NA          | EDL          | 0D8FF7E6      | f4995efc          | Qualcomm USB Composite Device:QCS_KALAMAP-HDK_SN:0D8FF7E6 |
+-----+-----+-----+-----+-----+-----+
```

Flashing Configuration Data Table Image

Before the Configuration Data Table (CDT) image can be flashed with PCAT, a few variables first need to be defined in the terminal.

```
# Set the Qualcomm board Serial Number seen from running 'pcat -DEVICES'
```

```
$ SERIAL_NUMBER="xxxxxxx"
```

```
# The directory custom build or '$BSP_DIR/prebuilt_release' for a release build
```

```
$ RELEASE_DIR="$BSP_DIR/qcs8550-le-1-0_amss_standard_oem_apqgps.git"
```

If this is the first time flashing the SBC, the CDT image must be flashed onto the device first before the software images are flashed onto the device.

Refer to 80-50450-3 Rev. AB (QCS8550.LE.1.0 Linux Embedded Platform Quick Start Guide) for more information about the CDT and how it can be generated.

```
$ tar -xf $BSP_DIR/CDT.tar.gz -C $BSP_DIR
$ pcat -PLUGIN SD -DEVICE $SERIAL_NUMBER -BUILD "$BSP_DIR/CDT" -DEVICEPROG
"$BSP_DIR/CDT/xb1_s_devprg_ns.melf" -RAWPROG "$BSP_DIR/CDT/rawprogram3.xml"
-PATCHPROG "$BSP_DIR/CDT/patch3.xml" -MEMORYTYPE UFS -ERASE True -RESET False
-SLOT 0
```

If successful, PCAT should say **"Downloading software images completed on the device..."**.

Flashing Software Images

```
$ pcat -PLUGIN SD -DEVICE $SERIAL_NUMBER -BUILD "$RELEASE_DIR/contents.xml"
-MEMORYTYPE UFS -FLAVOR asic -ERASE True -RESET True -SLOT 0 -FLATTEN False
```

If successful, PCAT should say **"Downloading software images completed on the device..."**.

Working with the board

Powering the Board

See the [datasheet](#) for power connection.

For access to the serial console, the USB to Serial adapter must be connected to the host machine. Once the board has finished booting, which takes about 10-15 seconds, the login prompt should appear in the serial console:

```
IMDT Linux Embedded distro targeting debug builds.  
LE.QCLINUX.0.0-11200-K2C.0-1-g161a3f6 imdt-qcs8550-sbc ttyMSM0  
  
imdt-qcs8550-sbc login:
```

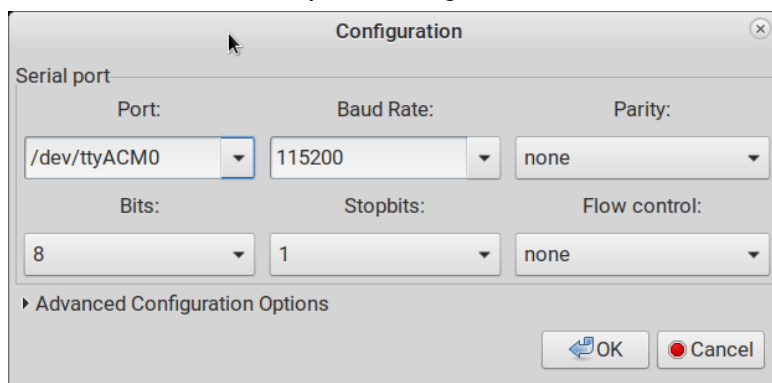
The username is **root**, and the password is **oelinux123**.

Connecting to the Debug Serial Consoles

When connected to a Linux host system, the serial ports will appear as USB devices (typically **/dev/ttyUSB0** and **/dev/ttyUSB1**). On Ubuntu systems, the serial consoles can be accessed using GTKTerm. Adding the current user to the **dialout** group will allow serial port access without requiring super-user privileges:

```
$ sudo apt install gtkterm  
$ sudo usermod -a -G dialout <username>  
# Log out and back in for the change to take effect
```

The screenshot below shows the serial port settings:



Android Debug Bridge (ADB) and Fastboot

ADB is a command line tool used to communicate with devices running the ADB daemon. It can be used to launch a shell on the host which can be used to execute commands on the device. Fastboot is a communication protocol that is used to communicate with a device in Fastboot mode. Fastboot mode can be used for reading and writing the flash contents of the device.

Setting up ADB and Fastboot

Install ADB and Fastboot on host machine

```
$ sudo apt-get install adb fastboot
```

Check connected devices with ADB

```
$ adb devices
```

Development Workflow using ADB

Once the device is connected to the ADB server on the host machine, shell commands can then be issued from the host machine to the device. The device filesystem can then be accessed using *adb push* or *pull*. And a terminal can be launched on the device using *adb shell*.

Remount the root directory for RW access

```
$ adb root
$ adb shell mount -o remount,rw /
$ adb unroot
```

Copying files onto device

```
$ adb push /path/to/application /data/
```

Launch ADB Shell

```
$ adb shell
```

Flashing using Fastboot

To flash the device using fastboot, the device has to be rebooted into Fastboot mode. This can be done through ADB.

Reboot device into Fastboot Mode

```
$ adb root
$ adb reboot bootloader
```

Flash whole image onto device using Fastboot Protocol

```
$ cd ${QCOM_ROOT_DIR}/common/build
$ python fastboot_complete.py
$ fastboot reboot
```

Note that if using the prebuilt release make sure QCOM_ROOT_DIR points to that folder instead. For example:

```
QCOM_ROOT_DIR=$BSP_DIR/imdt_qcs8550_sbc_prebuilt_release_v${BUILD_VERSION}
```

Using the Display

The following section describes how to connect IMDT's display to the QCS8550 SBC. At the moment, only the IMDT DSI display is supported. However, work is in progress to add support for HDMI displays as well via a DSI to HDMI adapter. For more information about display connection see the [datasheet](#).

To use the IMDT display (PN:IM-MSC-0006), IMDT's IM-PCA-0029 DSI adapter must be connected to the display using the display flex cables. Once connected, the adapter can then be connected to the SBC display connector as shown in the following image:

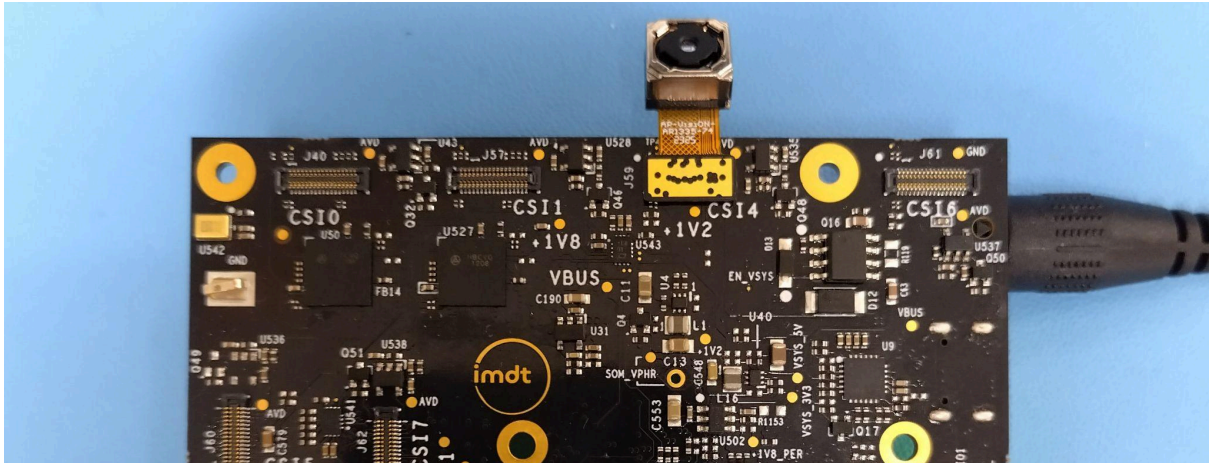


Once the board is powered, the display will then boot up with the Yocto Project splash screen. The touchscreen will also be working.

Streaming from MIPI Camera's

Streaming from AR1335

To stream from the AR1335 (PN:IM-CCM-0009) it must first be connected to the SBC board whilst the board is powered off. For example, here is an AR1335 connected to CSI4.



For additional information see the [datasheet](#) for connection details.

Once the AR1335 has been connected, the board can then be powered on. To stream to the display, from the AR1335, use the following GStreamer commands:

```
$ export WAYLAND_DISPLAY=wayland-1 && export XDG_RUNTIME_DIR=/run/user/root
$ gst-launch-1.0 -e qtiqmmfsrc camera=0 !
video/x-raw,width=1920,height=1080,format=NV12,framerate=30/1 ! waylandsink
fullscreen=true sync=false async=true
```

Audio

Audio files can be played on the SBC using headphones. This can be done in one of two ways:

- Using PulseAudio:

```
$ systemctl start pulseaudio
$ gst-launch-1.0 filesrc location=$WAV_FILE_PATH ! wavparse ! audioconvert
! pulsesink volume=6
```

- Using agmplay:

```
$ systemctl stop pulseaudio
$ agmplay $WAV_FILE_PATH -D 100 -d 100 -i MI2S-LPAIF-RX-PRIMARY
```

Please note that agmplay will not work while the pulseaudio process/service is running.

SD-Card

The board has an sd-card slot which can be used for external storage. Connected a sd-card and verify that the card is detected:

```
$ dmesg
[ 2636.277963][    T70] mmc1: new ultra high speed SDR104 SDHC card at
address aaaa
[ 2636.290379][    T70] mmcblk1: mmc1:aaaa SD32G 29.7 GiB
```

Check where the sd-card is mounted:

```
$ df -h | grep mmcblk1p1
/dev/mmcblk1p1      83.0M    22.1M    60.9M   27% /mnt/sdcard
```

Now you can read and write to the sd-card through */mnt/sdcard* folder.

Using the IMU (Accelerometer and Gyroscope)

The IMDT QCS8550 SBC has an onboard IMU (LSM6DSVQ) which features a 3-axis digital accelerometer and a 3-axis digital gyroscope. The IMU is connected to the ADSP core which

reduces the amount of processing to be done on the main CPU cores. Qualcomm has developed some tools to allow testing of sensors connected to the ADSP core.

To capture accelerometer data from the IMU with their test tool, run the following command:

```
$ ssc_drva_test -sensor=accel -duration=5 -sample_rate=480
```

To capture gyroscope data from the IMU with their test tool, run the following command:

```
$ ssc_drva_test -sensor=gyro -duration=5 -sample_rate=480
```

For more information about Qualcomm's Software stack for sensors located on the ADSP see: [Qualcomm Linux Sensors Guide](#)

Using the Temperature and Humidity Sensor

The IMDT QCS8550 SBC has an onboard temperature and humidity sensor (SHT40). The sensor is connected to the ADSP core which reduces the amount of processing to be done on the main CPU cores. Qualcomm has developed some tools to allow testing of sensors connected to the ADSP core.

To capture ambient temperature data with their test tool, run the following command:

```
$ ssc_drva_test -sensor=ambient_temperature -duration=5 -sample_rate=480
```

To capture humidity data with their test tool, run the following command:

```
$ ssc_drva_test -sensor=humidity -duration=5 -sample_rate=480
```

For more information about Qualcomm's Software stack for sensors located on the ADSP see: [Qualcomm Linux Sensors Guide](#)

Qualcomm's Intelligent Multimedia SDK

Qualcomm's Intelligent Multimedia SDK (IM SDK) includes a collection of GStreamer based AI demo applications that showcase the processing capabilities of their processors. Several of the demos in the IM-SDK can be run on the IMDT QCS8550 SBC. To run the AI demo applications in IM SDK, the relevant models must be downloaded from the internet and installed onto the target. The following section describes how to do this.

Downloading Qualcomm's GStreamer AI Models

Downloading Base Models and Labels

Method 1: Download directly to the device (if the device has internet access)

Qualcomm [instructs users](#) to download files using a script downloaded directly to the device that is connected to the internet.

First, run the following commands:

```
$ adb root
$ adb shell
# mount -o remount,rw /
```

Then follow [Qualcomm's instructions](#) to download the models and labels

After the downloads have completed, do:

```
# mv /etc/labels/* /opt/
# mv /etc/models/* /opt/
```

Method 2: Download to the Host machine (if the device has no internet access)

If the device does not have an internet connection, the script can be downloaded to the host machine and modified to make the output paths correct.

Firstly, save [Qualcomm's script](#).

Open it in a text editor, and replace this section from line 122 to 132:

```
if [ "$version" == "GA1.3-rel" ]; then
    outputmodelpath="$outputpath"
    outputlabelpath="$outputpath"
    outputconfigpath="$outputpath"
else
    outputmodelpath="/etc/models/"
    outputlabelpath="/etc/labels/"
    outputconfigpath="/etc/configs/"
    outputdatapath="/etc/data/"
    outputmediapath="/etc/media/"
fi
```

With:

```
outputmodelpath="./opt/"
outputlabelpath="./opt/"
outputconfigpath="./trash/"
outputdatapath="./trash/"
outputmediapath="./trash/"
```

Then run the script (for Windows in PowerShell) with the following options:

```
$ chmod +x download_artifacts.sh
$ ./download_artifacts.sh -v GA1.4-rel -c QCS8275
```

Once that has finished, execute the following:

```
$ cp ./opt/yolonas.labels ./opt/yolov8.labels
$ adb root
$ adb shell mount -o remount,rw /
$ adb push ./opt/* /opt
```

Downloading YOLOv8 model

The YOLOv8 model must be downloaded and built separately. Qualcomm's documentation on the YOLOv8 model can be found on [Qualcomm's YOLOv8 GitHub page](#).

Firstly, [install Miniconda](#) , [create a Qualcomm AI Hub Account](#), and get an AI Hub API token by going to Account -> Settings -> API Token, then do:
(only on Windows in PowerShell):

```
> & "$env:USERPROFILE\miniconda3\Scripts\conda.exe" init powershell
```

Then:

```
$ conda create python=3.10 -n qai_hub  
$ conda activate qai_hub  
$ pip install qai-hub  
$ qai-hub configure --api_token API_TOKEN  
$ pip install "qai-hub-models[yolov8-det]"
```

Once the downloads have completed, do:

```
$ python -m qai_hub_models.models.yolov8_det.export --quantize w8a8
```

Once the above steps have completed, use adb to push the models and labels to /opt.
Unix host machine:

```
$ adb root  
$ adb shell mount -o remount,rw /  
$ adb push build/yolov8_det/yolov8_det.tflite  
/opt/YOLOv8_Detection_Quantized.tflite  
$ adb push build/yolov8_det/yolov8_det.tflite  
/opt/yolov8_det_quantized.tflite
```

Then enter the adb shell:

```
$ adb shell
```

Running Qualcomm's GStreamer AI Sample Applications

Initialisation

Before running any of these applications, make sure that the following commands have been executed on the device:

```
# export WAYLAND_DISPLAY=wayland-1 && export XDG_RUNTIME_DIR=/run/user/root  
# mount -o remount,rw /
```

Classification

To run the Classification sample app, use this command:

```
$ gst-ai-classification
```

Daisychain Detection Pose

To run the Daisychain Detection Pose sample app, use this command:

```
$ gst-ai-daisychain-detection-pose
```

Multistream Inference

To run the Multistream Inference sample app, use this command:

```
$ gst-ai-multistream-inference --use-case=0  
--constants="YOLov8,q-offsets=<21.0, 0.0,  
0.0>,q-scales=<3.0546178817749023, 0.003793874057009816, 1.0>;" -d
```

Object Detection

To run the Object Detection sample app, use this command:

```
$ gst-ai-object-detection
```

Parallel Inference

To run the Parallel Inference sample app, use this command:

```
$ gst-ai-parallel-inference
```

Pose Detection

To run the Pose Detection sample app, use this command:

```
$ gst-ai-pose-detection
```

Networking

Gigabit Ethernet over RJ45

The Ethernet port on the SBC supports up to 1 Gigabit/s. There is currently no default configuration of the ethernet and thus it needs to be manually configured on the first boot.

There are currently two operational modes that the ethernet can be configured to operate in; DHCP client mode and DHCP server mode. When running in DHCP server mode, the SBC will run a DHCP server on the ethernet interface, which will allow the SBC to be connected directly to a computer. When in DHCP server mode, the SBC will assign IP addresses to devices connected to it. Also when operating as a DHCP server, the ethernet interface is configured to have a static IP address (172.16.30.100).

The Ethernet can also be configured in DHCP client mode. In DHCP client mode, the ethernet port can be connected to an existing network and get access to the internet if the network has internet access. In this mode, the ethernet interface runs a DHCP client and must be assigned an IP address from a router (or anything on the network running a DHCP server).

DHCP Client

Once the device boots, run the following commands to configure the ethernet interface as a DHCP client:

```
$ cat /run/machine-id > /etc/machine-id
$ mv /lib/systemd/network/80-enP1p1s0-dhcp-client.network.disabled
/lib/systemd/network/80-enP1p1s0-dhcp-client.network
$ systemctl restart systemd-networkd
```

DHCP Server

Once the device boots, run the following commands to configure the ethernet interface as DHCP server:

```
$ cat /run/machine-id > /etc/machine-id
$ mv /lib/systemd/network/80-enP1p1s0-dhcp-server.network.disabled
/lib/systemd/network/80-enP1p1s0-dhcp-server.network
$ systemctl restart systemd-networkd
```


To check if the device has been configured properly execute the following:

```
$ ifconfig
enP1p1s0  Link encap:Ethernet  HWaddr AA:0D:09:19:EA:10
          inet addr:192.168.50.227  Bcast:192.168.50.255
Mask:255.255.255.0
          inet6 addr: fe80::a80d:9ff:fe19:ea10/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:5494 errors:0 dropped:4650 overruns:0 frame:0
          TX packets:398 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:356956 (348.5 KiB)  TX bytes:26774 (26.1 KiB)
```

Note that in this case, the ethernet interface (enP1p1s0) has been configured in DHCP client mode as it has been assigned an IP address from a router.

Onboard WiFi

The onboard WiFi ([IW416](#)) can be configured and used for networking.

Once the board is booted, enable the wireless networking interface with following command:

```
$ ip link set wlan0 up
```

To connect the WiFi interface to a local password protected (WPA2/WPA3) wireless network, use wpa_supplicant. First open/create the wpa_supplicant configuration file with the following:

```
$ nano /etc/wpa_supplicant.conf
```

With the configuration file open, copy and paste the following into /etc/wpa_supplicant.conf (Replace **NETWORK_NAME** and **NETWORK_PASSWORD** with the network credentials and change the country code if not located in the UK):

```
country=gb
ctrl_interface=/var/run/wpa_supplicant
ctrl_interface_group=0
update_config=1
network={
    scan_ssid=1
    ssid="NETWORK_NAME"
    psk="NETWORK_PASSWORD"
}
```

To begin connecting to the wireless network, execute the following commands:

```
$ wpa_supplicant -i mlan0 -D nl80211 -c /etc/wpa_supplicant.conf &  
$ udhcpc -i mlan0
```

To test the network connection use the following commands:

```
$ networkctl status mlan0  
$ ping google.com
```

The `networkctl status` command should show an IP address for `mlan0` if connected to the local network. And the `ping` command will only work if connected to a local network that has access to the internet.

WiFi via M2.E WiFi card

The M.2 Key E connector can be used to connect M.2 Key E PCIe devices like NVME SSD drives and WiFi/Bluetooth cards. Supported alternative to the onboard wifi is to use an M.2 Key E WiFi card. In the current BSP release, support has been added for the [RTL8822CE](#) and [QCNFA765](#) M.2 Key E Wi-Fi module.

To use the WiFi card, first connect it to SBC on the M.2 Key E slot (See [Connectors - Bottom Side](#) for location of the M.2 Key E slot).

Once the WiFi card is connected, boot the board and enable the wireless networking interface with following command:

```
$ ip link set wlp1s0 up
```

To connect the WiFi interface to a local password protected (WPA2/WPA3) wireless network, use wpa_supplicant. First open/create the wpa_supplicant configuration file with the following:

```
$ nano /etc/wpa_supplicant.conf
```

With the configuration file open, copy and paste the following into `/etc/wpa_supplicant.conf` (Replace **NETWORK_NAME** and **NETWORK_PASSWORD** with the network credentials and change the country code if not located in the UK):

```
country=gb
ctrl_interface=/var/run/wpa_supplicant
ctrl_interface_group=0
update_config=1
network={
    scan_ssid=1
    ssid="NETWORK_NAME"
    psk="NETWORK_PASSWORD"
}
```

To begin connecting to the wireless network, execute the following commands:

```
$ wpa_supplicant -i wlp1s0 -D nl80211 -c /etc/wpa_supplicant.conf &  
$ udhcpc -i wlp1s0
```

To test the network connection use the following commands:

```
$ networkctl status wlp1s0  
$ ping google.com
```

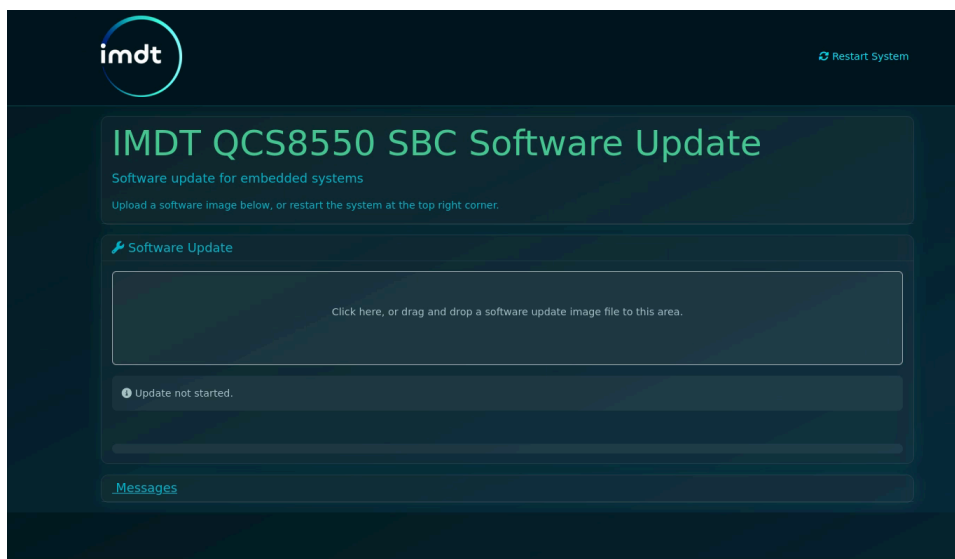
The `networkctl status` command should show an IP address for `wlp1s0` if connected to the local network. And the `ping` command will only work if connected to a local network that has access to the internet.

BSP OTA Software Update

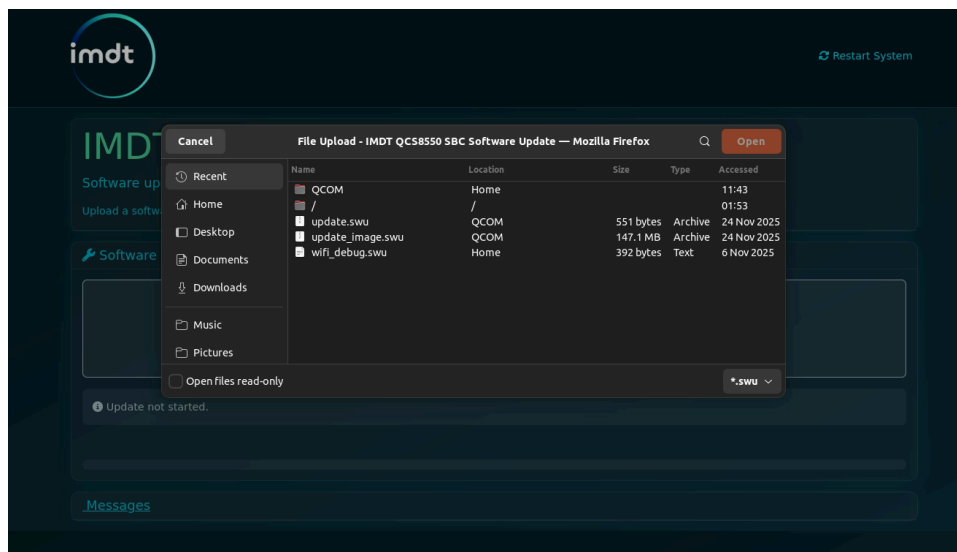
The BSP now includes SWUpdate which provides a way to update the BSP via a webserver. Assuming the board is connected to the network via WiFi or Ethernet (See [Networking](#) section), the SWUpdate web page can be accessed by going to this web address: <http://imdt-qcs8550-sbc:8080>

If two devices are connected to the network, the IP address of the device will need to be used instead of the hostname; for example 192.168.1.243:8080.

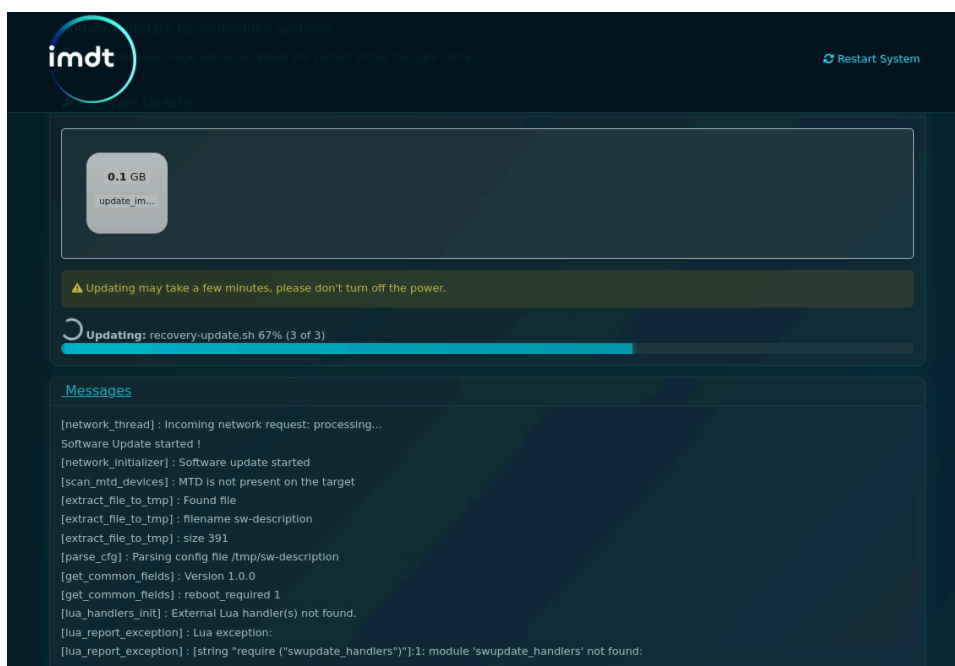
Once on the SWUpdate web page, the page should look similar to the image below.



To update the software, upload the SWU file by clicking on the Update section of the webpage. This should open a file browser as shown in the image below.



Within the file browser, select the SWU (.swu) image file. Once selected, the SWU file will be uploaded to the target and SWUpdate will begin to update the BSP. When the update is complete, the target will reboot.



To check if the target has successfully managed to update to a newer version, check the release version by running the following command:

```
$ adb shell lsb_release -a
```

The output should look something like this:

```
LSB Version:n/a
Distributor ID:  imdt-qcom-distro-debug
Description:IMDT Linux Embedded distro targeting debug builds.
IMDT-QCOM-BSP-2.0.0 92d3199
Release:      IMDT-QCOM-BSP-2.0.0-92d3199
Codename:     n/a
```

If the update fails for any reason, the board will reboot to the last working BSP version.

Using the SDK

The SDK consists of the following components:

- Cross-compilation toolchain for the QCS8550
- Libraries, headers, and symbols for the software packages that are installed in *imdt-image-weston*
- Environment setup script, which sets up the cross-development environment by defining variables and preparing for SDK use

Installing the SDK

The SDK can be installed on the host development machine by running the toolchain installer (`imdt-qcs8550-sbc-sdk-v${BUILD_VERSION}.sh`). The default installation directory is `/usr/local/imdt-qcom-distro-debug-x86_64`, but this can be changed as part of the installation process.

Note: The permissions may need to be changed on the toolchain installer to make it executable, and write permissions will be required for the toolchain installation directory.

```
$ mkdir -p /opt/yocto/sdks
$ chmod +x imdt-qcs8550-sbc-sdk-v${BUILD_VERSION}.sh
$ ./imdt-qcs8550-sbc-sdk-v${BUILD_VERSION}.sh
IMDT Linux Embedded distro targeting debug builds.
SDK installer version 0-1-g161a3f6
=====
Enter target directory for SDK (default: /usr/local/imdt-qcom-distro-debug-x86_64):
/opt/yocto/sdks
You are about to install the SDK to "/opt/yocto/sdks". Proceed [Y/n]? Y
Extracting
SDK.....done
Setting it up...done
SDK has been successfully set up and is ready to be used.
Each time you wish to use the SDK in a new shell session, you need to source the
environment setup script e.g.
$ . /opt/yocto/sdks/environment-setup-aarch64-oe-linux
$ . /opt/yocto/sdks/environment-setup-aarch64-oe-linux-sd11vm
```


Setting up a Development Environment

To set up a development environment, source the environment script in a terminal:

```
$ source /opt/yocto/sdks/environment-setup-aarch64-oe-linux
```

The GCC cross-compilation toolchain will now be available for use:

```
$ aarch64-oe-linux-gcc --version
aarch64-oe-linux-gcc (GCC) 11.4.0
Copyright (C) 2021 Free Software Foundation, Inc.
This is free software; see the source for copying conditions.  There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
```

Building a “Hello World” Application Using the SDK

This section describes the process of using the SDK to build a “Hello World” application for the board. Two files are required: a `main.cpp` C++ source file and a `CMakeLists.txt` file. The contents of these files are shown below:

main.cpp

```
#include <iostream>

int main(int argc, char *argv[])
{
    std::cout << "Hello world" << std::endl;
}
```

CMakeLists.txt

```
cmake_minimum_required(VERSION 3.19)
project(hello_world)
add_executable(hello_world main.cpp)
```

To build the application, source the environment script and run the following commands:

```
$ source /opt/yocto/sdks/environment-setup-aarch64-oe-linux
$ cmake .
$ cmake --build .
[ 50%] Building CXX object CMakeFiles/hello_world.dir/main.cpp.o
[100%] Linking CXX executable hello_world
[100%] Built target hello_world
```

Running the Example Application on the board

The example application can be transferred to the board using ADB from the host PC. Refer to the section on using ADB to see how to set it up.

```
$ adb wait-for-device
$ adb push hello_world /data/
$ adb shell
$ cd /data/
$ ./hello_world
Hello world
```

Appendix

1.1. Setting up a new user within docker

As mentioned earlier, if the host UID and GID do not match the existing user in the container, some issues may occur whilst trying to build the image. To avoid these issues, a new user with the same UID and GID as the host must be created within the docker container. To accomplish this, execute the following commands within the docker container (NOTE: User must export variables **HOST_UID** and **HOST_GID** before executing the following commands):

```
sudo su
export HOST_UID="PUT HOST UID HERE"
export HOST_GID="PUT HOST GID HERE"
sudo groupadd -g ${HOST_GID} host
sudo useradd host -u ${HOST_UID} -g ${HOST_GID} -m -s /bin/bash

sudo usermod -a -G host dev
sudo usermod -a -G dev host
echo "host ALL=(ALL) NOPASSWD:ALL" >> /etc/sudoers

newgrp host
newgrp dev

su host
cd /home/host/

ln -sf /home/dev/build_scripts /home/host/build_scripts
ln -sf /home/dev/downloads /home/host/downloads
ln -sf /home/dev/.pyenv /home/host/.pyenv
ln -sf /home/dev/.bin /home/host/.bin
ln -sf /home/dev/.cache /home/host/.cache

cp /home/dev/.bashrc /home/host/.bashrc
chown host:host /home/host/.bashrc

sudo chown host:host /home/host/.pyenv/version
sudo chown host:host /home/host/.pyenv/shims/

git config --global user.name "host"
git config --global user.email "host@imd-tec.com"
git config --global color.ui auto

source /home/host/.bashrc
```

1.2. Connectors - Top Side

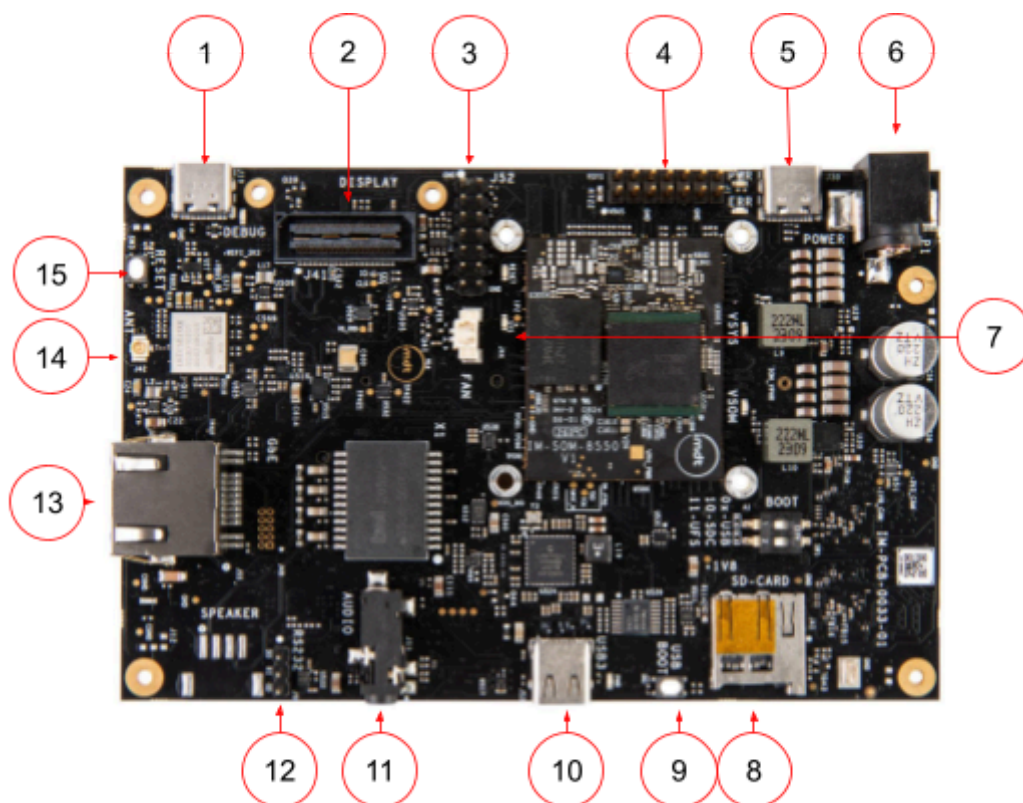


Figure 2 QCS SBC Getting Familiar top view

#	Functionality	Silk name
1	DEBUG	J19
2	Display	J41
3	Auxiliary	J52
4	PSM (Power supply Module)	J1
5	Power In - USB-PD	J30
6	Power In - DC Jack	P1
7	Fan	J44
8	Micro SD card	J43
9	USB BOOT Button	SW2
10	USB 3.0 C-Type	J12

11	3.5mm Audio Jack	J11
12	RS-232	J9
13	Ethernet	J55
14	u.FL BLE antenna	J42
15	Reset Button	SW3

1.3. Connectors - Bottom Side

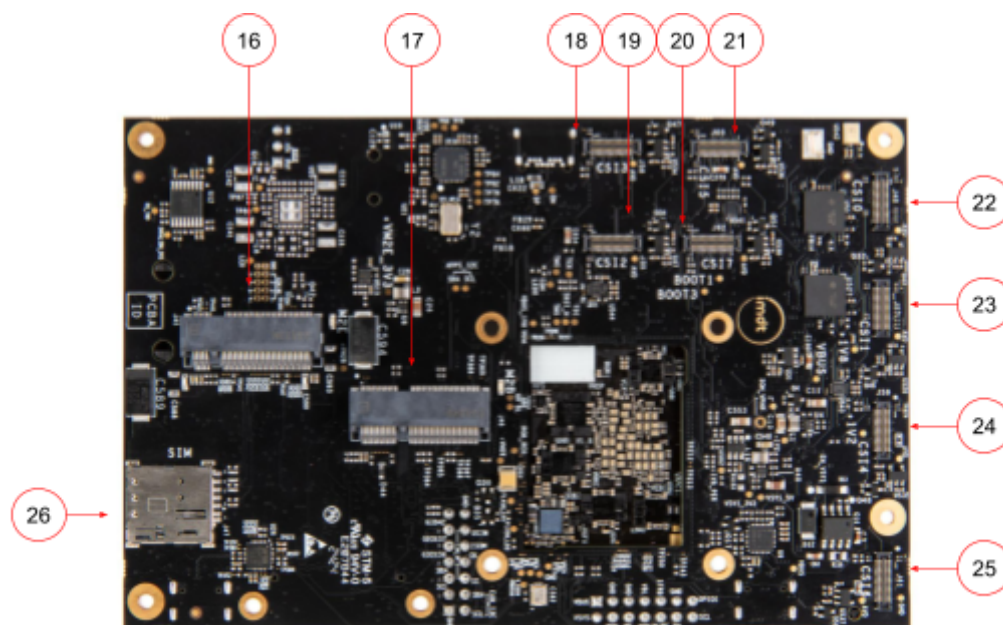


Figure 3 QCS SBC Getting Familiar bottom view

#	Functionality	Silk name
16	M.2 Key-E	J45
17	M.2 Key-B	J46
18	CSI 3 connector	J58
19	CSI 2 connector	J35
20	CSI 7 connector	J62
21	CSI 5 connector	J60
22	CSI 0 connector	J40
23	CSI 1 connector	J57
24	CSI 4 connector	J59
25	CSI 6 connector	J61
26	Nano SIM card	J47